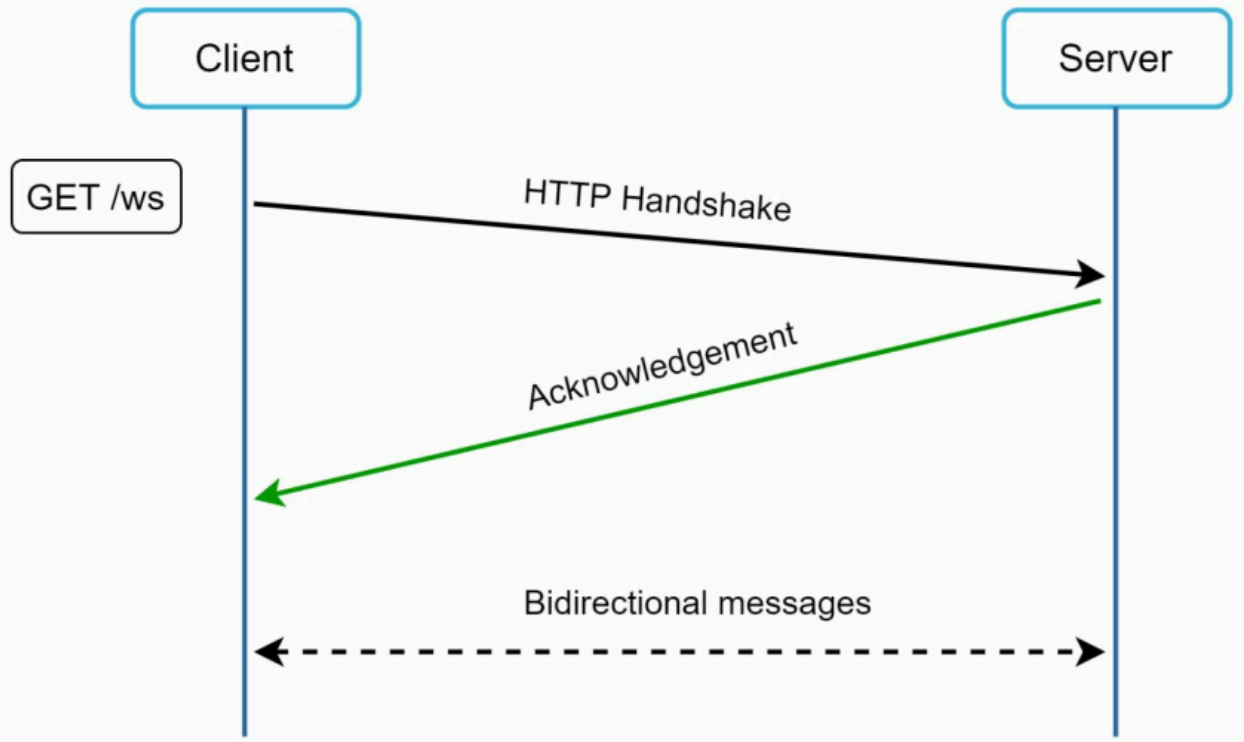
WebSocket

WebSocket is the most common modern solution for sending **real-time asynchronous updates** between server and client.

Unlike polling or long polling, WebSocket creates a **persistent full-duplex connection**, allowing both client and server to send data anytime.



As shown in **Figure**, the connection is initiated by the client and upgraded from HTTP to WebSocket.

1. Core Idea

Traditional HTTP works like this:

```
None
Client sends request

Server sends response

Connection ends
```

WebSocket works like this:

```
None
Client connects once

Connection stays open

Client ↔ Server exchange data anytime
```

This enables true real-time communication.

2. How WebSocket Works

Step 1: HTTP Handshake

Client first opens a normal HTTP request:

```
None
GET /chat HTTP/1.1

Upgrade: websocket

Connection: Upgrade
```

Server responds:

```
None
101 Switching Protocols

Upgrade: websocket
```

Now HTTP is upgraded into a WebSocket connection.

Step 2: Persistent Connection

After handshake:

```
None
Client ↔ Server
```

No repeated reconnections needed.

3. Diagram

```
None
Client ---- HTTP Handshake ----> Server

Client <--- 101 Upgrade ------- Server


Persistent WebSocket Open


Client <========> Server

Send anytime     Send anytime
```

4. Bidirectional Communication

This is the biggest advantage.

Client → Server

```
None
Send chat message

Typing status

Game move

User action
```

Server → Client

```
None

New messages

Live notifications

Price updates

Presence updates
```

5. Why WebSocket is Better Than Polling

✔ No Repeated Requests

Polling:

```
None
Request every 5 sec
```

WebSocket:

```
None
One connection stays open
```

✔ Low Latency

Updates are pushed instantly.

```
None
Message sent → instantly received
```

✔ Lower Bandwidth Overhead

No repeated HTTP headers every request.

✔ Real-Time UX

Best for live experiences.

6. Firewall Friendly

WebSockets usually use:

```
None
Port 80  (HTTP)

Port 443 (HTTPS / WSS)
```

Since these are standard web ports, WebSockets generally work behind corporate firewalls and NAT.

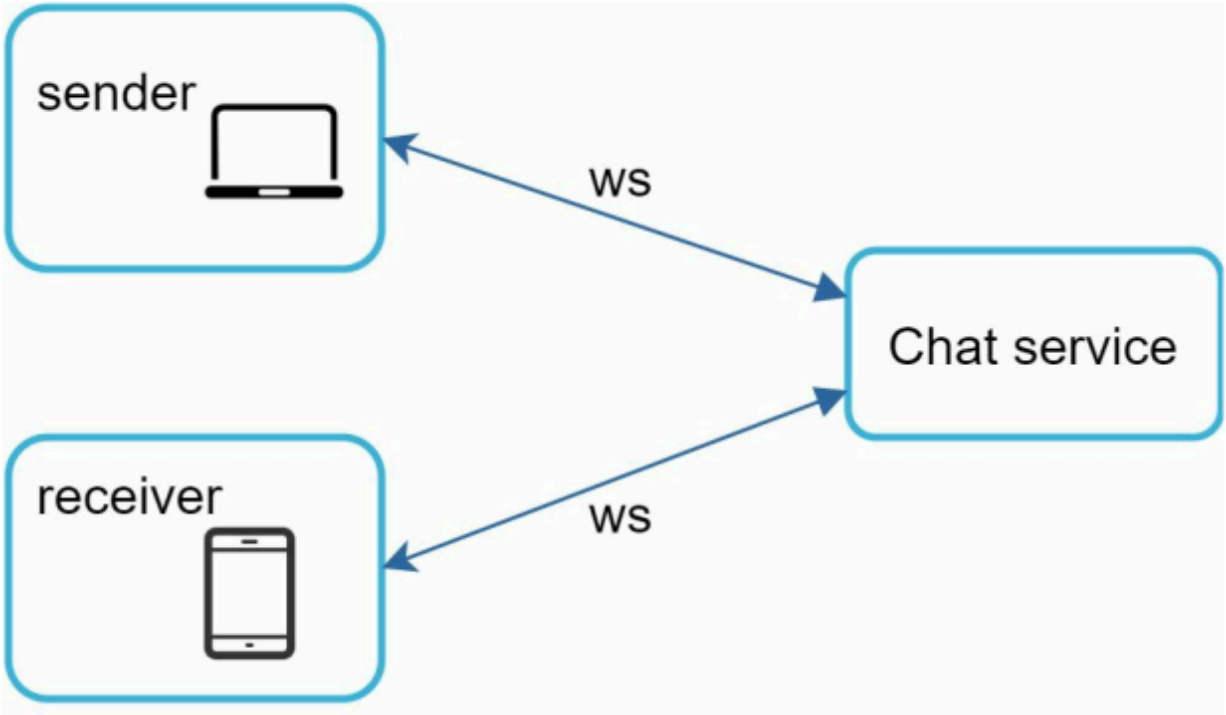
7. Using WebSocket for Both Sender & Receiver

Earlier systems used:

```
None
Send message via HTTP POST

Receive via polling
```

Modern systems use WebSocket for both:



As shown in **Figure**

```
None
Client ↔ WebSocket ↔ Server
```

Benefits:

- Simpler architecture
- Single protocol
- Easier client code
- Lower latency both ways

8. Server-Side Challenges

Persistent connections improve UX but create operational challenges.

Connection Management

Millions of users = millions of open sockets.

Need to manage:

```
Memory per connection

Heartbeats

Idle timeouts

Reconnect logic
```

Horizontal Scaling

User connected to Server A may receive message from Server B.

Need shared backend:

```
None
Redis Pub/Sub

Kafka

RabbitMQ

Message Broker
```

Load Balancing

Often need:

```
None
Sticky sessions

Connection-aware load balancer
```

9. WebSocket Architecture Example

```
None
Users

  ↓

Load Balancer

  ↓

WebSocket Servers

  ↓

Redis Pub/Sub / Kafka

  ↓

Databases
```

10. Use Cases

WebSocket is ideal for:

```
Chat apps

WhatsApp / Messenger

Live gaming

Trading dashboards

Collaborative editors

Live sports score apps

Ride tracking

IoT control panels
```

11. Comparison Table

Feature	Polling	Long Polling	WebSocket
Repeated Requests	High	Medium	No
Real-time Speed	Low	Medium	High
Bidirectional	No	No	Yes

Persistent Connection	No	Temporary	Yes
Best for Chat	Poor	Medium	Excellent

12. Interview Insight

If interviewer asks:

Why WebSocket for chat system?

Answer:

```
None
WebSocket provides persistent bidirectional communication,

low latency message delivery, reduced HTTP overhead,

and real-time updates, making it ideal for chat systems.
```

13. Important Production Considerations

Heartbeats / Ping-Pong

Detect dead clients.

```
None
ping → pong
```

Reconnect Strategy

If network drops:

```
Retry with exponential backoff
```

Authentication

Use:

```
JWT

Session token

Secure WSS
```

14. One-Line Summary

WebSocket is a persistent bidirectional communication protocol upgraded from HTTP that enables real-time low-latency messaging between client and server, making it the preferred solution for chat and live systems.